

Introduction

This unit discusses as such, C programming does not provide direct support for error handling but being a system programming language, it provides you access at lower level in form of return values.

Most of the C or even Unix function calls return -1 or NULL in case of any error and set an error code errno. It is set as a global variable and indicates an error occurred during any function call. You can find various error codes defined in header file. So a C programmer can check returned values and can take appropriate action depending on return value.

It is a good practice to set errno to 0 at the time of initializing a program. A value of 0 indicates that there is no error in the program.



Error Handling in 'C'

Error handling is a big part of writing software, and when it is done poorly, software becomes difficult to extend and to maintain. Programming languages like C++ or Java provide exceptions and destructors that make error handling easier. Such mechanisms are not natively available for C, and literature on good error handling in C is widely scattered over the internet.

Errors are problems or faults that occur in a program, which makes the behavior of the program abnormal, and experienced developers can also make these faults. Programming errors are also known as the bugs or faults, and the process of removing these bugs is known as debugging.



Identify



Analyze

Check errno for error codes



Resolve



Detect errors through return values

Take appropriate action based on error

2/5

NSError

C programs use NSError objects to convey information about runtime errors that users need to be informed about. In most cases, a program displays this error information in a dialog or sheet. But it may also interpret information and either ask the user to attempt to recover from the error or attempt to correct the error on its own.



NSError Object consists of:



Domain

The Error domain can be one of predefined NSError domains an arbitrary string describing a custom domain and domain must not be nil.



Code

The error code for error.



User Info

The User Info dictionary for error and user Info may be nil.



Example:

```
NSString *domain = @"com.MyCompany.MyApplication.ErrorDomain";
NSString *desc = NSLocalizedString(@"Unable to complete process", @"");
NSDictionary *userInfo = @{@"NSLocalizedStringKey" : desc };
NSError *error = [NSError errorWithDomain:domain code:101 userInfo:userInfo];
```



Pattern for Error Handling

A collected knowledge on good error handling in the form of C error-handling patterns and a running example that applies patterns. The patterns provide good practice design decisions and elaborate on when to apply them and which consequences they bring. For a programmer, these patterns remove the burden of making many fine-grained decisions. Instead, a programmer can rely on knowledge presented in these patterns and use them as a starting point to write good code.

Figure: An overview of patterns and their relationships



Pattern_Name	Description
Function Split	The function has several responsibilities, which makes the function hard to read and maintain. Therefore, split it up. Take a part of a function that seems useful on its own, create a new function with that, and call that function.

Guard Clause

The function is hard to read and maintain because it mixes pre-condition checks with the main program logic of the function. Therefore, check whether you have mandatory pre-conditions and immediately return from the function if these pre-conditions are not met.

Samurai Principle

When returning error information, you assume that caller checks for this information. However, the caller can simply omit this check and the error might go unnoticed. Therefore, return from a function victorious or not at all. If there is a situation for which you know that an error cannot be handled, then abort the program.

Goto Error Handling

Code gets difficult to read and maintain if it acquires and cleans up multiple resources at different places within a function. Therefore, have all resource cleanup and error handling at the end of the function. If a resource cannot be acquired, use goto statement to jump to resource cleanup code.

Cleanup Record

It is difficult to make a piece of code easy to read and maintain if this code acquires and cleans up multiple resources, particularly if those resources depend on one another. Therefore, call resource acquisition functions as long as they succeed, and store which functions require cleanup. Call cleanup functions depending on these stored values.

Object-Based Error Handling

Having multiple responsibilities in one function, such as resource acquisition, resource cleanup, and usage of that resource, makes that code difficult to implement, read, maintain, and test. Therefore, put initialization and cleanup into separate functions, similar to the concept of constructors and destructors in object-oriented programming.



Errno Values and Meaning

errno is a global variable indicating an error occurred during any function call and it is defined inside header file. When a function is called in C, a variable named errno is automatically assigned a code (value) which can be used to identify the type of error that has been encountered. Different codes values for errno mean different types of errors.



Common errno Values:

error value	Error
1	Operation not permitted
2	No such file or directory
3	No such process

4	Interrupted system call
5	I/O error
6	No such device or address
7	The argument list is too long
8	Exec format error
9	Bad file number
10	No child processes
11	Try again
12	Out of memory
13	Permission denied



Example:


```
#include <errno.h>
#include <stdio.h>

int main()
{
    // If a file is opened which does not exist,
    // then it will be an error and corresponding
    // errno value will be set
    FILE* fp;
    // opening a file which does not exist
    fp = fopen("Error Handling.txt", "r");
    printf("Value of errno: %d\n", errno);
    return 0;
}
```



Key Takeaway

While C doesn't provide built-in exception handling like other languages, it offers mechanisms like return values and errno to detect and handle errors effectively. Using proper error handling patterns is essential for creating robust and maintainable C programs.

